

TP2

Utilisation de clefs , un cas pratique : SSH et Kerberos

Introduction à Kerberos

Le système d'authentification Kerberos est celui utilisé à l'IUT'O. Dans ce tp, nous allons tenter de mieux comprendre son fonctionnement.

Commencer par étudier la page wikipédia sur ce protocole ([https://fr.wikipedia.org/wiki/Kerberos_\(protocole\)](https://fr.wikipedia.org/wiki/Kerberos_(protocole))) puis répondez aux questions suivantes:

1. Où a été développé le protocole Kerberos?
2. Kerberos est-il un protocole symétrique ou asymétrique?
3. En vous aidant de la commande `klist` exécutée sur les machines du département, identifiez le royaume "realm" auquel votre machine appartient ainsi que le nom d'hôte.
4. Un exemple de service nécessitant une authentification kerberos pour y accéder est le serveur NFS permettant de monter vos répertoires personnels sur les machines du département. Résumez les différentes étapes nécessaires afin d'obtenir le ticket permettant d'accéder à ce service.

Bilan

Une fois qu'un client s'est identifié, celui-ci obtient un ticket. Le ticket joue le rôle d'une carte d'identité à péremption assez courte, huit heures généralement. Si nécessaire, celui-ci peut être annulé prématurément. Sous les systèmes Kerberos comme celui du MIT, cette procédure est généralement appelée via la commande « `kdestroy` ».

Premières manipulations

Afficher les tickets

Vous pouvez voir votre ticket avec la commande `klist`.

```
$ klist
Credentials cache: API:1769E1B8-6A49-4CB8-8172-379613D4C42F
Principal: oXXXXXXXX@IUT45.UNIV-ORLEANS.FR

Issued                Expires                Principal
Oct 24 15:31:21 2023  Oct 25 01:31:18 2023  krbtgt/IUT45.UNIV-ORLEANS.FR@IUT45.UNIV-ORLEANS.FR
```

Lorsque vous possédez ce ticket vous pouvez vous en servir pour vous connecter sur une autre machine de l'IUT'O et en plus cette machine exploite ce ticket pour monter votre home. Par conséquent vous avez alors accès à votre home depuis une autre machine.

Vous pouvez essayer avec le client SSH de vous connecter à distance sur la machine de votre voisin. Celui-ci peut alors voir votre connexion avec la commande `who`

Exemple avec la machine `or-iut-i004-f03`:

```
# sur la machine f04
oXXXXXXXX@or-iut-i004-f04:~$ ssh oXXXXXXXX@or-iut-i004-f03
# sur la machine f03
oXXXXXXXX@or-iut-i004-f03:~$ who
```

1. Faites les manipulations précédentes en adaptant avec les noms de vos machines.

En principe vous n'avez pas à fournir de mot de passe car vous avez déjà le ticket valide. C'est comme si vous rentriez dans une salle de concert avec un ticket que vous avez déjà acheté et présenté à la sécurité du spectacle.

Rebonds

Vous pouvez comme cela rebondir de machines en machines, mais il est plus fiable d'ajouter `-K` pour spécifier que l'on souhaite déléguer à la machine distante les droits de l'utilisateur fin d'accéder à ses fichiers (son home) , cela se passe avec la délégation Kerberos :

```
oXXXXXXXX@or-iut-i004-f02:~$ ssh -K oXXXXXXXX@or-iut-i004-f03
oXXXXXXXX@or-iut-i004-f03:~$
```

Pour sortir de tous ces shell distants vous pouvez utiliser la commande `exit` ou bien tout simplement `CTRL-D`

Attention!

Sans le `-K` la plupart du temps cela marche aussi mais parfois la délégation ne se fait pas sans que les chargés de système à l'IUT'O ne sachent vraiment pourquoi. Aussi si vous rencontrez l'erreur suivante lors de la connexion:

```
Could not chdir to home directory /home/iut45/Etudiants/oXXXXXXXX: Permission denied
```

C'est que vous avez réussi à vous connecter à la machine distante **MAIS** que cette dernière n'a pas reçu la délégation Kerberos et donc n'a pas de ticket pour pouvoir prouver au serveur NFS que vous pouvez monter votre home.

Dans ce cas soit vous recommencez en précisant le `-K` , soit vous tapez dans la session distante: `kinit` afin de valider un ticket Kerberos mais alors vous devez fournir à la machine distante votre mot de passe.

Comment détruire ses tickets?

La commande `kdestroy -A` détruit vos tickets.

Testez cela et voyez que vous ne pouvez plus vous connecter à distance aussi facilement. A présent il vous faut utiliser votre mot de passe. Mais une fois celui-ci fourni, vous avez un ticket qui est généré et vous êtes à même de monter votre home personnel et vos fichiers sont bien présents.

Vous pouvez aussi spécifier à votre client SSH que vous ne souhaitez pas utiliser de système d'authentification autre que Kerberos. À cette fin utilisez l'option: `-o PreferredAuthentications=gssapi-with-mic`

```
ssh -o PreferredAuthentications=gssapi-with-mic -K oXXXXXXXX@or-iut-i004-f04
```

Si vous avez détruit votre ticket vous ne pouvez plus vous connecter via SSH:

```
$ kdestroy -A
$ ssh -o PreferredAuthentications=gssapi-with-mic oXXXXXXXX@or-iut-i004-f04
oXXXXXXXX@or-iut-i004-f04: Permission denied (publickey,gssapi-with-mic,password).
```

Mais vous pouvez redemander un ticket via la commande `kinit`

```
$ kinit
Password for oXXXXXXXX@IUT45.UNIV-ORLEANS.FR:
```

Alors vous pouvez vous connectez de nouveau via Kerberos:

```
oXXXXXXXX@or-iut-i004-f03:~$ ssh -o PreferredAuthentications=gssapi-with-mic \  
-K oXXXXXXXX@or-iut-i004-f04  
oXXXXXXXX@or-iut-i004-f04:~$
```

Vous devez normalement obtenir la sortie suivante

```
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-49-generic x86_64)  
  
0 updates can be applied immediately  
  
*** System restart required ***  
oXXXXXXXX@or-iut-i004-f04:~$
```

Sans Kerberos : les mots de passe classiques

Tous les systèmes ne sont pas fondés sur Kerberos. Ce dernier est réservé aux grandes institutions (comme l'IUT'O n'est-ce-pas ...). Assez classiquement on peut simplement faire reposer la connexion sur l'usage de mots de passe stocké encryptés sur la machine.

Vous pouvez utiliser ce système en précisant cela à SSH. C'est souvent le cas par défaut mais à l'IUT'O, par défaut c'est Kerberos.

```
ssh -o PreferredAuthentications=password oXXXXXXXX@or-iut-i004-f04
```

Notez que cela revient à ne pas utiliser Kerberos comme dans l'exemple précédent.

Votre home est tout de même monté car lorsque vous vous connectez à la machine un ticket Kerberos est immédiatement créé. Vérifiez cela avec `klist`

SSH et RSA

Vous aurez sans doute noté que Kerberos n'utilise que des chiffrements à **chefs symétriques**. Et pour cause! Il a été inventé dans les années 1980 **alors que** les chiffrements asymétriques émergeaient seulement.

Nous allons à présent passer par l'authentification via l'usage de clefs asymétriques [RSA](#).

Création d'une clef rsa

Il est aisé de créer une clef rsa utilisable pour se connecter avec ssh. Il suffit d'utiliser l'utilitaire `ssh-keygen` comme suit:

```
$ ssh-keygen -t rsa -f .ssh/id_rsa_test
```

Votre clef est privée, elle est alors stockée dans le fichier `.ssh/id_rsa_test` et votre clef publique est dans `.ssh/id_rsa_test.pub`

Vous pouvez y jeter un coup d'oeil:

```
$ cat .ssh/id_rsa_test
```

La clef publique est encodée dans un propre à ssh. C'est un fichier binaire qui est encodé via un encodeur binaire vers ascii nommé base64, ce qui est commode pour manipuler/copier du binaire.

Le clef publique peut être dérivée de la clef privée mais OpenSSH utilise un format qui lui est, là encore, propre et **ce n'est pas** le format PEM. Le format PEM est utilisé par OpenSSL. Le format est le PEM correspond à la traduction base64 des clefs à la norme x509 ASN.1.

Vous pouvez du reste le constater en listant le fichier : `.ssh/id_rsa_test.pub`

Exécuter les commandes suivantes:

```
ssh-keygen -f .ssh/id_rsa_test -y > test.pub
diff test.pub .ssh/id_rsa_test.pub
```

1. Que constatez-vous?

Utilisation de la clef

Afin d'utiliser cette clef pour se connecter à une machine à distance, il suffit de dire à cette machine qu'elle doit accepter une connexion sans mot de passe à la condition que l'utilisateur possède la clef privée d'un couple (clef privée, clef publique). Et l'élégance du système est là: il suffit de donner à la machine distante la clef **publique** de ce couple. Le protocole est basé sur le fait que seule la clef publique peut décrypter ce qui a été encrypté avec la clef privée.

Pour dire à la machine d'accepter un couple (clef privée, clef publique), il suffit de noter la clef publique dans le fichier `.ssh/authorized_keys`

Attention le nom du répertoire est important c'est `.ssh` car c'est bien là que ssh va chercher les clefs publiques autorisées.

Cela se fait souvent via une première connexion où on est obligé d'utiliser les mots de passe. Ou alors c'est l'administrateur qui met en place cela. Dans ce cas vous n'avez même pas besoin de mots de passe pour vous connecter à distance. Mais attention à ne pas perdre la clef publique ou pire, à se la faire dérober!

Voici un exemple où l'on envoie la clef publique sur la machine distante:

- Création d'un répertoire `~/ssh` (en forçant l'usage du **mot de passe** afin de ne pas utiliser Kerberos) sur la machine distante `or-iut-i004-f01`

```
oXXXXX@or-iut-i004-f04:~$ ssh -o PreferredAuthentications=password oXXXXX@or-iut-i004-f01 \
oXXXXX@or-iut-i004-f01:~$ mkdir -p /home/iut45/Etudiants/oXXXXX/.ssh
oXXXXX@or-iut-i004-f01:~$ exit
déconnexion
Connection to or-iut-i004-f01 closed
oXXXXX@or-iut-i004-f04:~$
```

- Copie de la clef publique dans le fichier `authorized_keys` de la machine distante (en forçant l'usage du **mot de passe** afin de ne pas utiliser Kerberos)

```
scp -o PreferredAuthentications=password .ssh/id_rsa_test.pub \
oXXXXX@or-iut-i004-f01:/home/iut45/Etudiants/oXXXXX/.ssh/authorized_keys
```

- Test de connexion (en forçant l'usage de la **clef** afin de ne pas utiliser Kerberos)

```
ssh -o PreferredAuthentications=publickey -i .ssh/id_rsa_test oXXXXX@or-iut-i004-f01
```

Envoi des clés publique via ssh-copy-id

Il est possible d'automatiser tout le processus d'envoi des clefs en utilisant l'utilitaire `ssh-copy-id` qui fait ce travail tout seul (en forçant l'usage du **mot de passe** afin de ne pas utiliser Kerberos)

```
ssh-copy-id -o PreferredAuthentications=password oXXXXX@or-iut-i004-f01
```

Attention!

La manipulation précédente utilisant `ssh-copy-id` n'est pas toujours fonctionnelle, par exemple lors du passage par des rebonds cela ne marche pas bien. Dans ce cas il faut passer par `scp`. Mais alors pour éviter d'écraser le fichier `authorized_keys` mieux vaut compliquer un peu la commande:

```
cat ~/.ssh/id_rsa.pub | ssh -o PreferredAuthentications=password \  
oXXXXX@info23-01 "umask 077 && mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys"
```

Mais en fait les transfert des clefs à distance n'est pas très utile à l'IUT'O!

1. Pourquoi?

Rebonds (tunnel ssh)

Vous pouvez à présent rebondir de machine en machines via l'usage de rebonds. Cela consiste à aller de machines en machines en passant par des machines où on a placé des clefs publiques. Cela permet par exemple d'accéder à des machines derrière des protections réseau. Mais il faut avoir au moins une machine accessible sans ces protections. C'est souvent une machine qui se trouve dans la partie "bastion" du réseau. Une partie exposée au réseau internet mais qui est plus surveillée et sécurisée. C'est le point d'accès pour les ssh depuis l'extérieur.

Par exemple je peux me connecter à info23-03 en rebondissant depuis info23-02. L'option `-J` permet d'établir un tunnel ssh qui transmet le flux de données en passant par une machine intermédiaire. Ici c'est info23-02 qui établit un tunnel vers info23-03.

Pour cela testons sans Kerberos

```
kdestroy -A  
ssh -J oXXXXXXXX@info23-02 oXXXXXXXX@info23-03
```

Ici on nous demande des mots de passe puisqu'il n'y a plus de ticket Kerberos à cause de la commande `kdestroy -A` que nous avons lancée juste avant.

Recommençons mais cette fois avec Kerberos:

```
kinit  
ssh -J oXXXXXXXX@info23-02 oXXXXXXXX@info23-04
```

Cela fonctionne mais il est difficile de s'en convaincre. Afin d'en être sûr, il est possible de visualiser les processus ssh à distance via ssh sur la machine rebond:

```
ssh -K oXXXXXXXX@info23-03 ps aux |grep ssh
```

Vous pouvez faire plusieurs fois la commande de dessus avec et sans la connexion de rebond et vous voyez que certains processus apparaissent lorsqu'il y a rebond.

Il est aussi possible d'enchaîner les rebonds. Ici commençons par enlever le ticket Kerberos. Trois fois le mots de passe est demandé.

```
kdestroy -A  
ssh -J oXXXXXXXX@info23-02,oXXXXXXXX@info23-03 oXXXXXXXX@info23-04
```

Ci-dessus nous sommes passé par 2 machines avant d'atteindre info23-04.

Avec Kerberos on peut enchaîner les rebonds sans mots de passe:

```
kinit
ssh -J oXXXXXXXX@info23-02,oXXXXXXXX@info23-03 oXXXXXXXX@info23-04
```

On peut aussi utiliser rsa pour la connexion finale

```
ssh -J oXXXXXXXX@info23-02 -o PreferredAuthentications=publickey \
-i ~/.ssh/id_rsa_test oXXXXXXXX@info23-04
```

Par contre un mot de passe nous est demandé pour nous connecter à la machine de rebond. La clef publique n'est utilisée que pour la **connexion finale**! C'est très peu pratique. C'est pourquoi l'usage n'est pas de passer par une commande `-J` mais plutôt par un fichier de configuration qui permet de spécifier comment sont accessibles les rebonds (et plein d'autres paramètres comme par exemple les ports à utiliser pour le tunnel ssh).

Il faut créer un fichier `.ssh/config` avec le droits 600.

Dans ce fichier placez:

```
Host info_distant
HostName info23-04
User oXXXXXXXX
ProxyJump info23-03
IdentityFile ~/.ssh/id_rsa_test
```

Si vous avez un ticket vous pouvez tester directement:

```
ssh info_distant
```

Cela marche car le tunnel passant par info23-03 utilise Kerberos.

Par contre si vous détruisez le ticket alors on vous demande un mot de passe pour le rebond sur info23-03, mais pas pour l'accès à info23-04, qui lui utilise la clef rsa.

Compliquons un peu notre fichier de configuration afin d'utiliser la clef rsa aussi pour la connexion au rebond:

```
Host info_distant
HostName info23-04
User oXXXXXXXX
ProxyJump machine_rebond
IdentityFile ~/.ssh/id_rsa_test

Host machine_rebond
HostName info23-03
User oXXXXXXXX
IdentityFile ~/.ssh/id_rsa_test
```

Testons sans Kerberos:

```
kdestroy -A
ssh info_distant
```

Cela marche! on a utilisé une machine rebond. On se connecte dessus (dans cet exemple info23-03) avec une clef rsa afin de créer un tunnel ssh vers la machine finale (dans cet exemple info23_04) sur laquelle on se connecte avec la même clef rsa.

En pratique à l'IUT'O c'est assez inutile de faire cela car toutes les machines sont identiques et ont accès à votre home et donc vos fichiers. Mais il peut arriver que vous ayez besoin d'accéder à une grosse machine distante pour

faire des calculs lourds par exemple. Ou bien que vous ayez besoins de fichiers sur une machine distante. Ces machines ne seront en générale accessible que via un VPN ou bien via ssh en passant par un rebond. Grâce à l'usage d'un fichier config et des clefs rsa vous pouvez accéder à ces machines distantes comme si elles étaient devant vous et sans paramétrage complexe ni usage de mots de passes.